Intro to Databases

1. What is a Database (vs. Spreadsheet / Text File)?

A **database** is an organized collection of structured data stored electronically in a computer system, optimized for fast retrieval, concurrent access, and data integrity.

Text Files (`.txt`, `.json`, `.csv`): Best for config or raw data export. Lack query engines, indexing, multi-user locks, and require full sequential scans to search.

Spreadsheets (Excel, Sheets): Built for human analysis and manual entry. Unsuited for complex data relationships, slow at scale, and prone to corruption under simultaneous automated writes.

Databases: Built for software applications. Provide high concurrency, ACID transactions, data validation, fast indexed lookups, and multi-user access control.

2. Types of Databases

Relational (SQL) vs. NoSQL

Database Type	Structure	Best For	Example Use Case	Popular Engines
Relational (SQL)	Tables (Rows & Columns)	Data integrity, strict schema, complex queries	**Banking Systems:** Atomic transactions (subtracting/adding funds without loss).	PostgreSQL, MySQL, Oracle
Document (NoSQL)	JSON/BSON Documents	Flexible schemas, semi-structured data	**E-Commerce Catalogs:** Storing products with varying attributes (e.g., shirts vs. laptops).	MongoDB, CouchDB

Key-Value (NoSQL)	Key-Value Pairs	Ultra-fast simple read/write lookups	**Session Caching:** Storing logged-in user tokens or temporary shopping carts.	Redis, Memcached
Wide-Column (NoSQL)	Dynamic Column Families	Massive write volumes, big data analytics	**IoT Telemetry:** Logging real-time sensor streams from millions of devices.	Cassandra, HBase
Graph (NoSQL)	Nodes & Edges (Connections)	Highly interconnected networks	**Social Networks:** Querying mutual friends or building recommendation engines.	Neo4j, Amazon Neptune

Core Difference

- **Relational (SQL):** Scales **vertically** (bigger server), prioritizes **ACID consistency** and strict structure.
- **NoSQL:** Scales **horizontally** (more servers), prioritizes **speed, flexibility**, and high throughput.

3. Hands-on: run a real database locally

Relational Power: Learned why apps use SQL databases instead of spreadsheets—for heavy multi-user concurrency, strict data validation, and fast queries.

Containerization: Ran PostgreSQL isolated inside Docker, mapping local port `5432` to the . hgvcontainerized database engine.

Client Tooling: Used DBeaver to connect to `localhost:5432`, inspect database structures, and run SQL scripts.

SQL Mechanics: Wrote DDL to build schemas (`CREATE TABLE`), DML to seed records (`INSERT`), targeted filters (`WHERE`), and summary metrics (`GROUP BY`, `COUNT`, `AVG`).

What does a DBMS do?

A **Database Management System (DBMS)** is the software layer that sits between the end-user/application and the underlying physical database. It handles administrative operations including:

Data Management: Executing queries to create, read, update, and delete data (CRUD operations).

Concurrency Control: Allowing thousands of concurrent read/write operations without data corruption.

Security & Access Control: Enforcing user permissions, roles, and encryption.
Data Integrity & Schema Enforcement: Ensuring data conforms to defined data types, constraints, and relationships.

Backup & Recovery: Preventing data loss during system crashes using log transaction management (e.g., WAL in PostgreSQL)

Questions to ask

Does the NoSQL involves code like how the PostgresSQL does in Bdeaver?

After running a successful PostgresSQL code in BDeaver do I need to run Postgres in Docker each time to see the code?